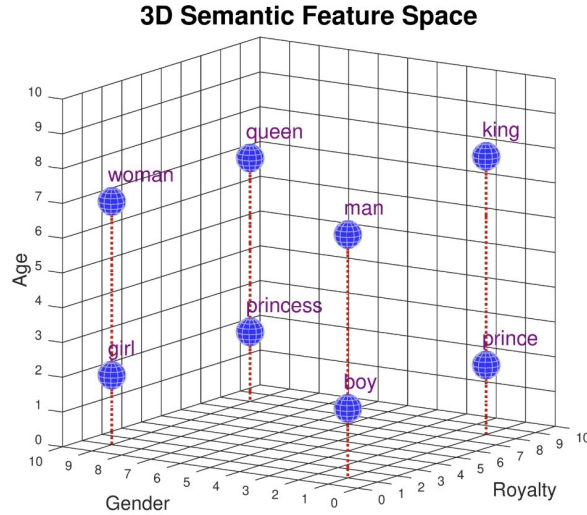
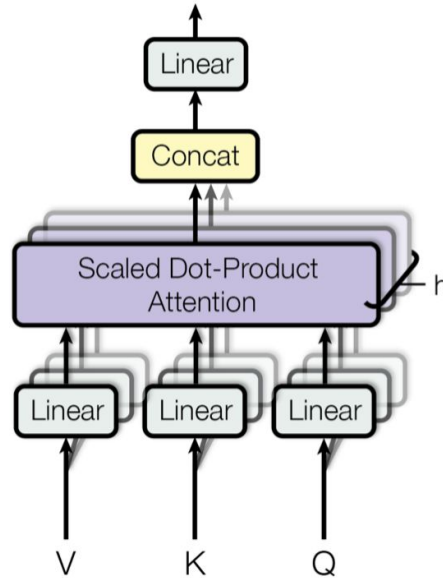


What I cannot create, I do not understand

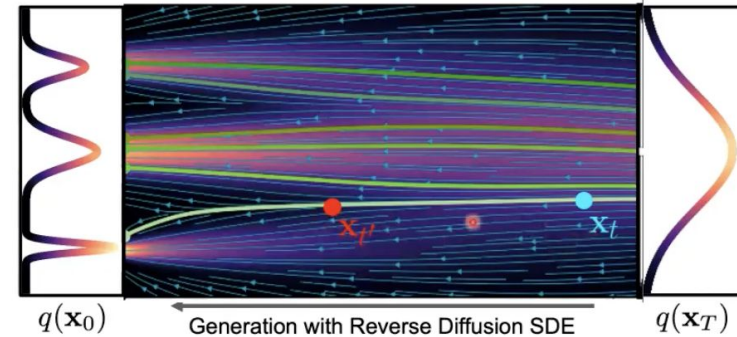
– GenAI and its application in Business School



The Language



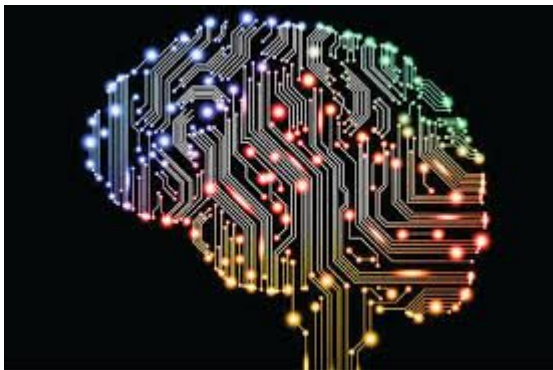
The Bard



The Artist

The structure of the pre

Deep Learning 101 -> 3 real applications -> discussions



Generative Model

Embedding & Representation Learning

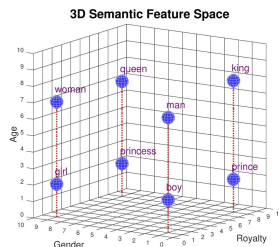
Neural Network

Multilayer Perceptron

Training

-> LLM

-> Diffusion Model



Crash Narratives

by William N. Goetzmann, Dasol Kim & Robert J. Shiller (2022)



Geoeconomic Pressure

by Christopher Clayton, Antonio Coppola, Matteo Maggiori & Jesse Schreger (2025)



Letting Logos Speak: Leveraging Multiview Representation Learning for Data-Driven Logo Design

by Ryan Dew, Asim Ansari, Olivier Toubia (2022)

Why do we care this fancy damn

Efficient-Market Hypothesis (EMH) - learn from Haowei Last Night

Efficient market levels

Semistrong-form efficiency

- ❖ Rate changes of the past as well as all known and available information are reflected in the market price

- ❖ E.g. information provided by media, private information services, investment advisors, annual balance sheets, annual reports, etc.

- ❖ It is not possible to gain over-average returns systematically by using a fundamental share analysis

- ❖ Current prices fully reflect all publicly available information including annual reports, news items, etc.

Theory:

Once information/methods are widely known, they get priced in.

This explains why quantitative company profit.

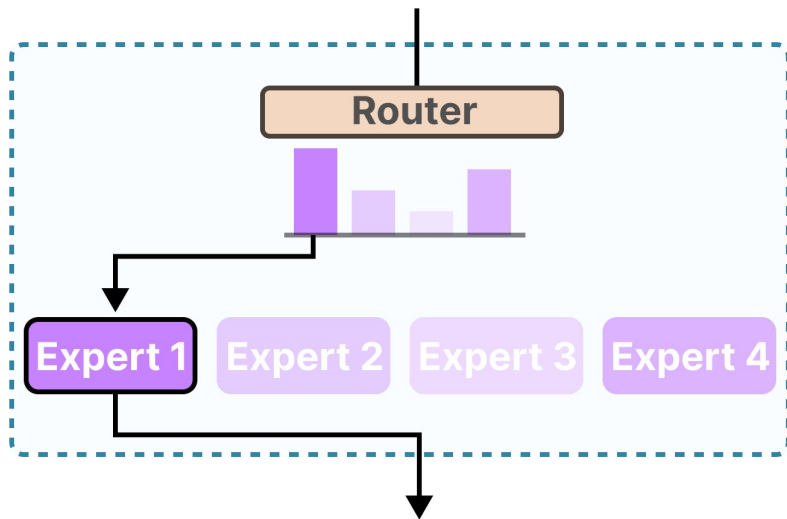
Everybody knows K-line chart ->
K-line chart does not work

Everybody knows Regression ->
Regression does not work

BUT No everyone knows ML,
lots of edge there

Why do we care this fancy damn

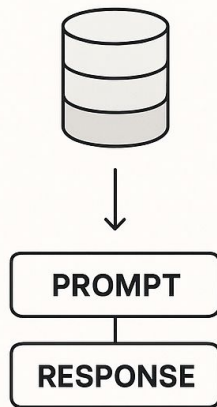
Social Science idea matter - MOE (Mixture of Expert) & Reinforcement Learning



**4 X 100B small Models beats 400B big Model
OB? OR? Game theory? Auditability?**

Model	Active Params	Arc-c	French HellaS	MMLU
LLaMA 1 33B	33B	39.3%	68.1%	49.9%
LLaMA 2 70B	70B	49.9%	72.5%	64.3%
Mixtral 8x7B	12B	58.2%	77.4%	70.9%

PRETRAINING



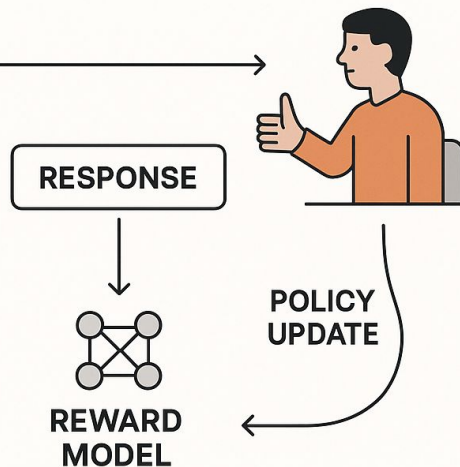
SUPERVISED
FINE-TUNING

RLHF



Deepseek R1

RLHF



**Better preference / utility estimation
Econ? Incentive? Causal Inference?**

Generative Model - toward a better understanding

Discriminative $\subseteq$ Generative

Given an image X , Y = label:



Discriminative Model

-> can only give YES/NO answer.

-> $P(Y|X) = P(\text{cat}|\text{this image}) = 0.99$

Generative Model (e.g. GPT)

-> tell you YES/NO answer too

-> You can do a lot more

-> $P(X|Y) = P(\text{new image} | \text{cat})$
 $= P(X,Y) / P(Y)$

-> Y can also be "prompt", think prompt as a more informative version of label, use to describe the X (data).

Main takeaways:

Discriminative $\subseteq$ Generative

Generative models are strictly more general

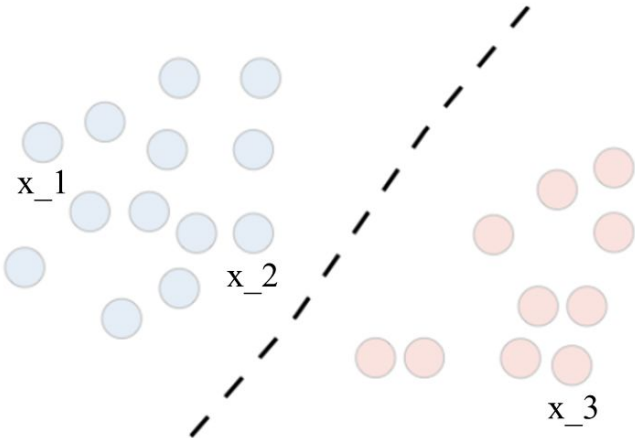
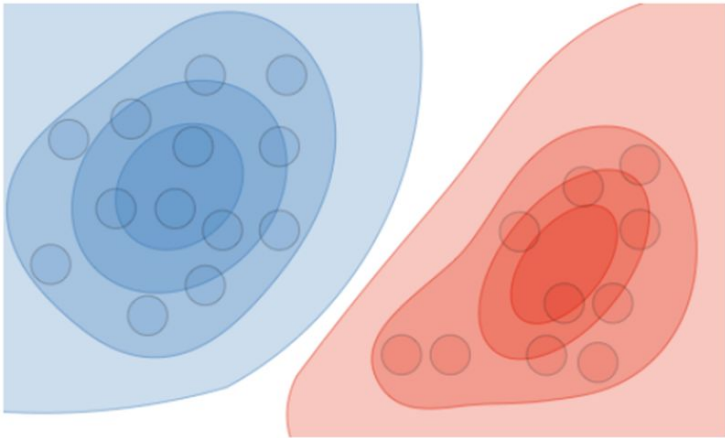
Discriminative model vs. Generative model

Visualization for binary classification task

$$P(y) = \sum_x P(x, y)$$

$$P(x | y) = \frac{P(x, y)}{P(y)}$$

$$P(y | x) = \frac{P(x, y)}{P(x)}$$

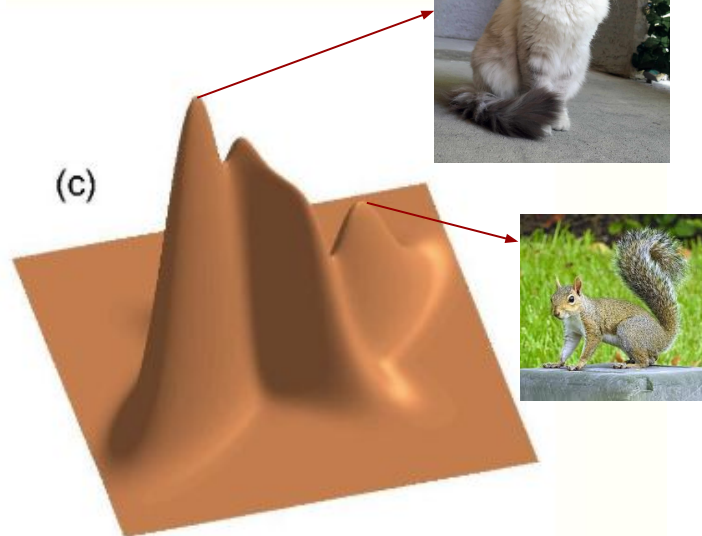
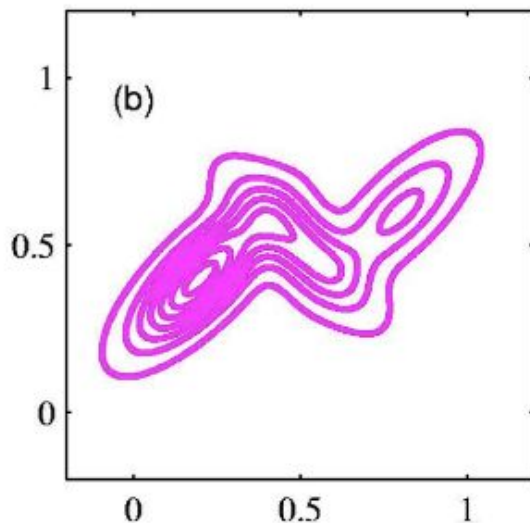
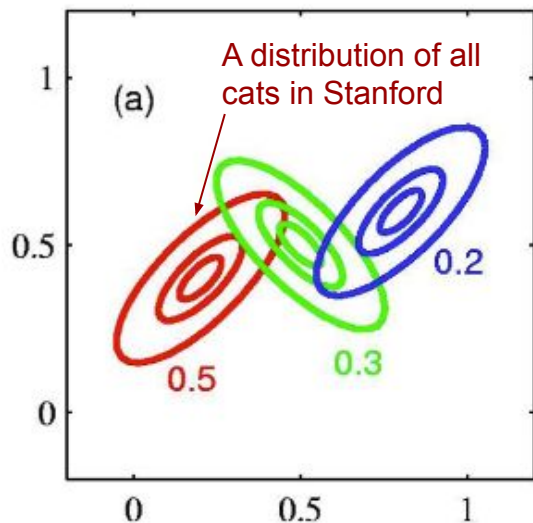
	Discriminative model	Generative model
Goal	Directly estimate $P(y x)$	Estimate $P(x y)$ to then deduce $P(y x)$
What's learned	Decision boundary	Probability distributions of the data $p_{\theta}(x_t y)$ and $p_{\theta}(x_t)$
Illustration		
Examples	Regressions, SVMs	GDA, Naive Bayes

Why learning distribution == generative

Use GMM (Gaussian Mixture Model) to learn Stanford Animals Classification (K = 3) ragdoll

$$P(x, y) = \sum_{k=1}^3 \pi_k \mathcal{N}(x, y \mid \mu_k, \Sigma_k) \quad P(X) = \sum_y P(X \mid Y=y) P(Y=y).$$

Sampling In high-D space



$Y = \{\text{cat, dog, squirrel}\}$

$P(X|Y = \text{cat})$ what cats are look like

$P(Y = \text{squirrel}) = 0.2$ how common

Combine all the animals, we get the **overall data distribution, unconditioned.**

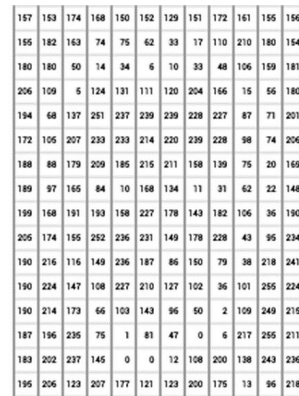
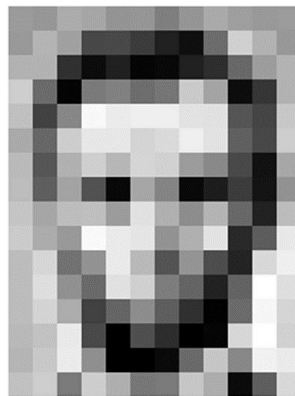
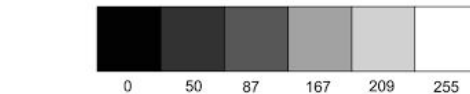
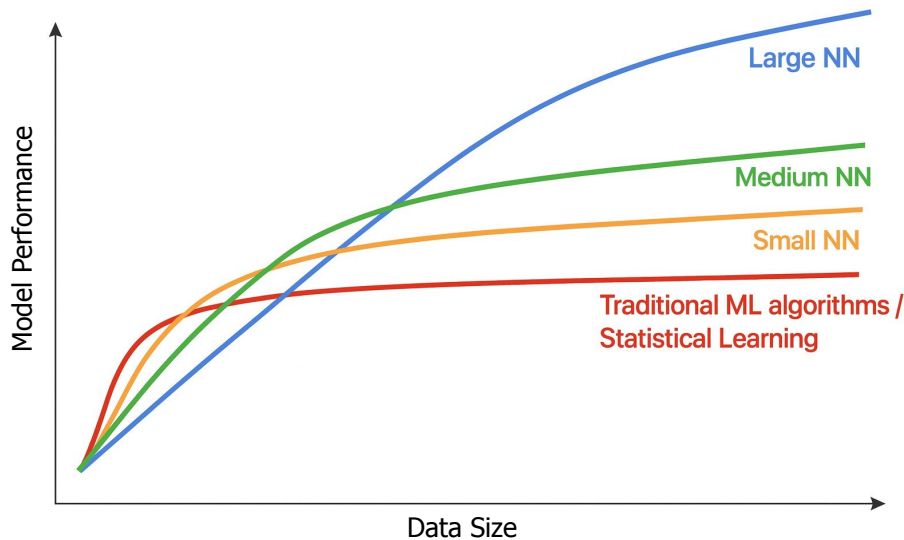
Peaks = typical animal (popular breed)
Ask "Show the **most typical** Animal in Stanford!"
Gen Model returns an animal in the Peak point

Scaling Law

Why we use Neural Network (NN) - powerful, flexible

Traditional CV:

Handcrafted features
(distances/angles of eye corners, nose tip, chin ...)



$f_{\theta}(X) \mapsto$ embedding vector in $\mathbb{R}^d$

Why NN is better than traditional ML model (none Neural Network models)

- NN has many more **parameters** and has capability to learn **complicated nonlinear patterns**.
- Also, we can expect the model to pick the most helpful features **automatically** by itself without too much expert-involved manual feature engineering.
- NO GIGO (data quality), NO Feature Engineering (N of dim of X), NO bias variance tradeoff (No overfit)

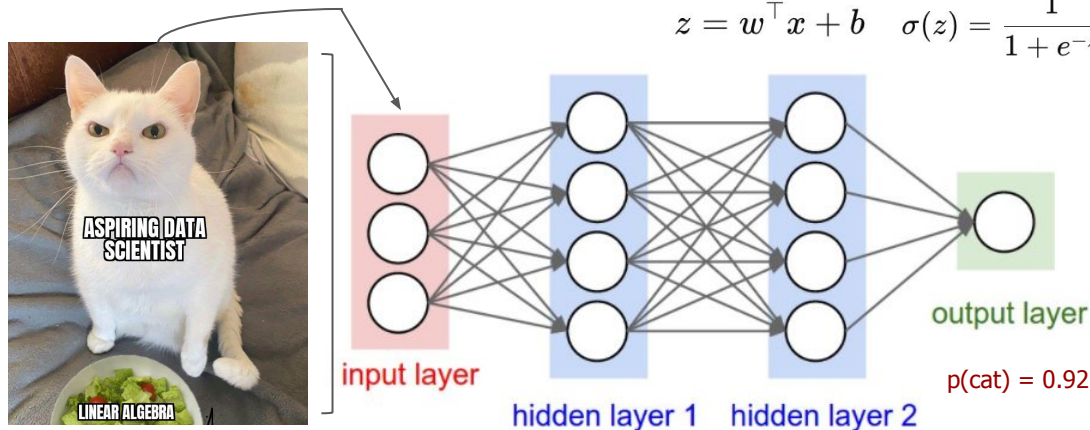
Neural Network / Multilayer Perceptron 101

understand the structure of NN, and model weights

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$\sum_{i=1}^K \text{softmax}(z_i) = 1$$

The Forward Path (Visual):



out input, a vector of pixels

The Forward Path (Linear Algebra):

Input layer X

hidden layer 1

hidden layer 2

output layer

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$

$$W_1 = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \\ w_{41}^{(1)} & w_{42}^{(1)} & w_{43}^{(1)} \end{bmatrix} \in \mathbb{R}^{4 \times 3}$$

$$h_1 = \phi(W_1 x) \in \mathbb{R}^{4 \times 1}$$

$$W_2 = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} & w_{14}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} & w_{24}^{(2)} \\ w_{31}^{(2)} & w_{32}^{(2)} & w_{33}^{(2)} & w_{34}^{(2)} \\ w_{41}^{(2)} & w_{42}^{(2)} & w_{43}^{(2)} & w_{44}^{(2)} \end{bmatrix} \in \mathbb{R}^{4 \times 4}$$

$$h_2 = \phi(W_2 h_1) \in \mathbb{R}^{4 \times 1}$$

$$W_3 = \begin{bmatrix} w_{11}^{(3)} & w_{12}^{(3)} & w_{13}^{(3)} & w_{14}^{(3)} \end{bmatrix} \in \mathbb{R}^{1 \times 4}$$

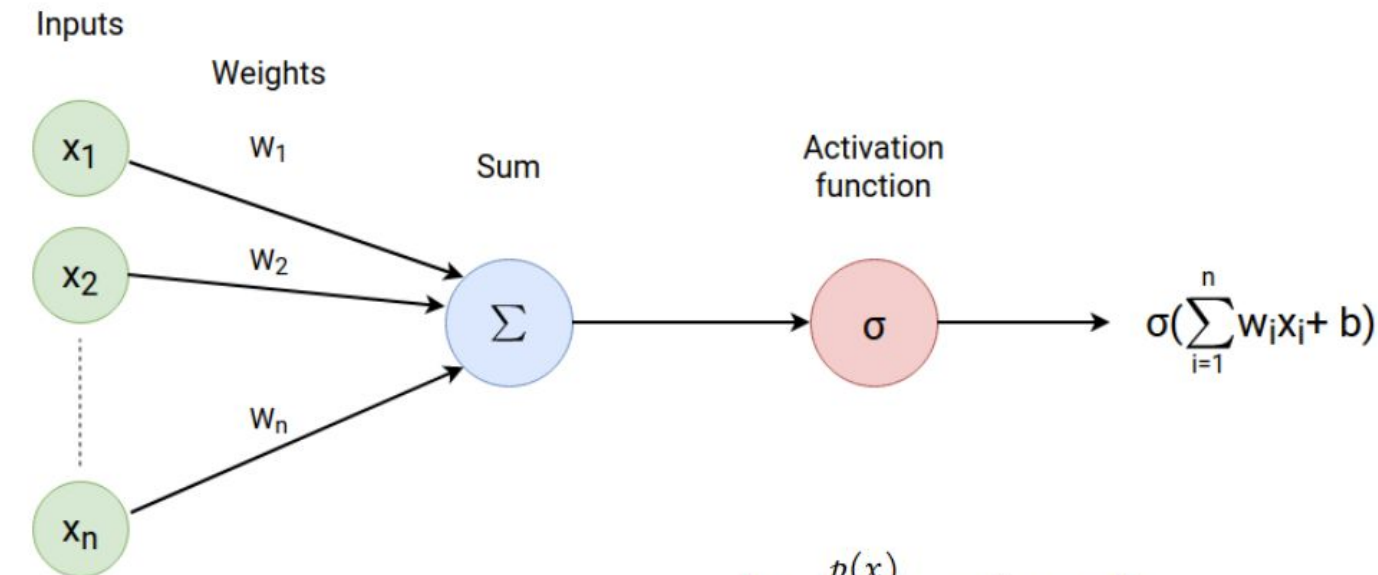
$$y = \psi(W_3 h_2) \in \mathbb{R}^{1 \times 1}$$

$$y = \psi(W_3 \phi(W_2 \phi(W_1 x)))$$

$$\begin{aligned} p(\text{cat}) &= 0.88 \\ p(\text{dog}) &= 0.12 \\ [0.88, 0.12]^T \end{aligned}$$

Neural Network / Multilayer Perceptron 101

What is Perceptron - logistic regression



$$\sigma(\sum_i^n \mathbf{w}_i x_i + b) = \sigma(\mathbf{w}^T x + b)$$

sigmoid: $\frac{1}{1+e^{-x}}$

apply sigmoid inside the network

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + x \cdot \beta$$

$$p(x; b, w) = \frac{e^{\beta_0 + x \cdot \beta}}{1 + e^{\beta_0 + x \cdot \beta}} = \frac{1}{1 + e^{-(\beta_0 + x \cdot \beta)}}$$

apply sigmoid to the outside, only one time

How to train your dragon - feedback

the GPA of classification model -> loss function, minimize entropy

$$y_i \sim \text{Bernoulli}(p_i)$$

$$\Pr(X=1) = p,$$

$$\Pr(X=0) = q = 1 - p.$$

$$f(k; p) = p^k (1 - p)^{1-k} \quad \text{for } k \in \{0, 1\}$$

Goal: Find the best S-shape line

(the shape is determined by weight & bias)

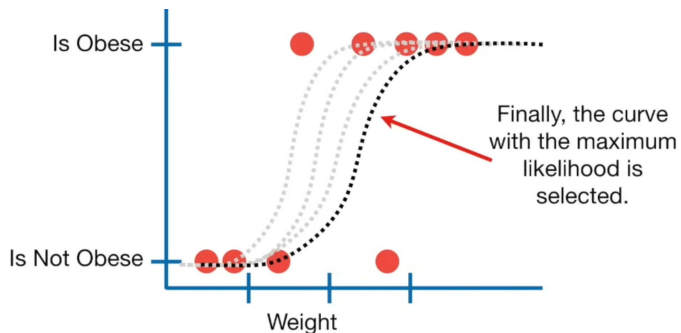
We want to measure **how far** the model's prediction

$$\hat{y} = \sigma(w \cdot x + b)$$

is from the **true label** $y \in \{0, 1\}$.

We define a **loss function** to quantify this difference:

$L(\hat{y}, y)$ = how much the prediction $\hat{y}$ differs from the true y .



How do we quantify curve whether the line is good / bad, GPA function

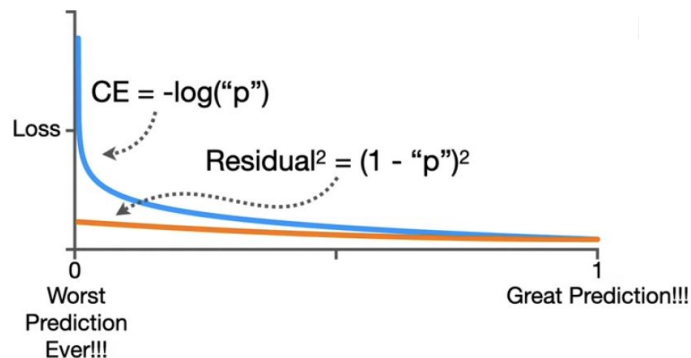
$$p(y | x) = L(\beta_0, \beta) = \prod_{i=1}^n \underline{p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}}.$$

Meaning of likelihood (predictive probability):

Given my current model parameters (β_0, β) , how plausible are the data I actually observed?

The loss function tell us how "bad" our prediction is - testing score.

- The simplest loss function is residual ($y_{\text{hat}} - y$)



Maximize likelihood == Minimize Cross Entropy

$$\mathcal{L}(\beta_0, \beta) = -\ell(\beta_0, \beta) = -\sum_{i=1}^n \left[y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i)) \right].$$

Intuition: Entropy (level of uncertainty)

Given a cat image X_1 , we know it's a cat so $Y = 1$

- Model 1: $P(\text{cat}|X_1) = 0.6$, $P(\text{not cat}|X_1) = 0.4$ -> likelihood = 0.6

- Model 2: $P(\text{cat}|X_1) = 0.9$, $P(\text{not cat}|X_1) = 0.1$ -> likelihood = 0.9

Obviously Model 2 is better, maximum likelihood work!

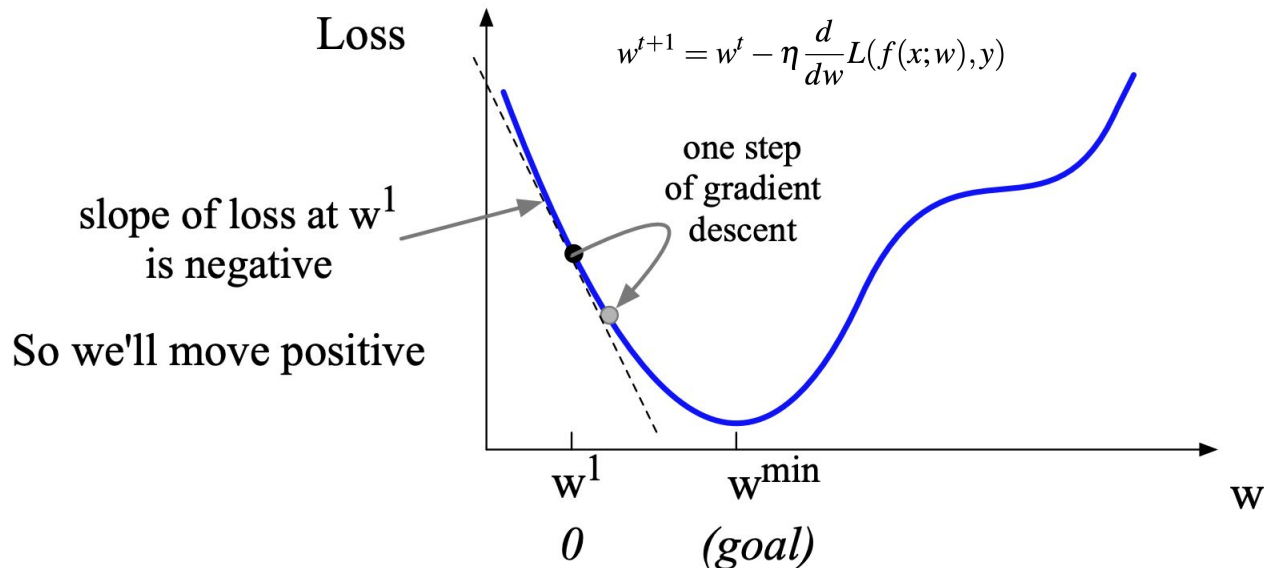
How to train your dragon - Learning

How to update the weight

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)}) \quad \text{logistic} \rightarrow \text{convex optimization (has just one minimum)}$$

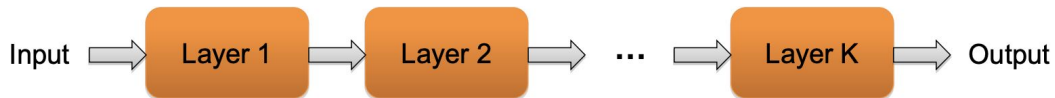
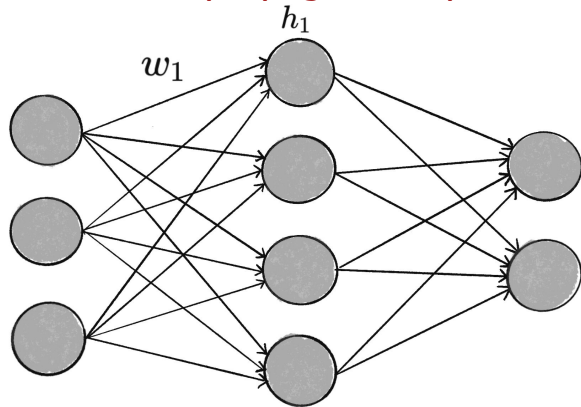
Q: Given current w , should we make it bigger or smaller?

A: Move w in the reverse direction from the slope of the function

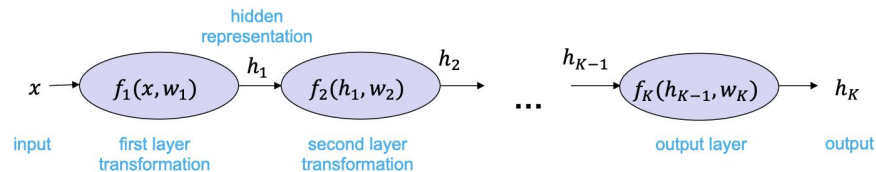


Update weight in context of Neural Network

Backpropagation - pass error by chain rule; update each layer with SGD (Optional)



The function computed by the network is a composition of the functions computed by individual layers (e.g., linear layers and nonlinearities):



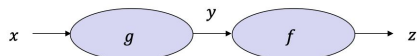
What is the SGD update for the parameters w_k of the k th layer?

$$w_k \leftarrow w_k - \eta \frac{\partial e}{\partial w_k}$$

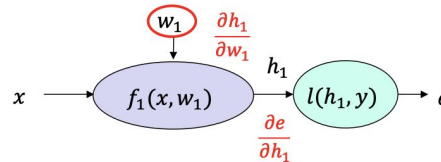
To train the network, we need to find the **gradient of the error w.r.t. the parameters of each layer**, $\frac{\partial e}{\partial w_k}$

Answer: the chain rule

$$h'(x) = f'(g(x))g'(x). \quad \frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$



$k = 1$



$$e = \ell(f_1(x, w_1), y), \quad h_1 = f_1(x, w_1)$$

$$\frac{\partial e}{\partial w_1} = \frac{\partial e}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

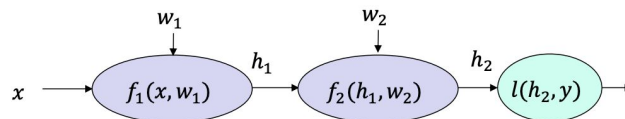
$$w_1 \leftarrow w_1 - \eta \frac{\partial e}{\partial w_1}$$

The intuition of gradient:

"how much changing this weight would reduce or increase the error."

- If I nudge w_1 , how does h_1 change? ($\partial h_1 / \partial w_1$)
- And if h_1 changes, how does the loss e change? ($\partial e / \partial h_1$)

$k = 2$



$$e = \ell(f_2(f_1(x, w_1), w_2), y), \quad h_2 = f_2(h_1, w_2)$$

$$\frac{\partial e}{\partial w_2} = \frac{\partial e}{\partial h_2} \cdot \frac{\partial h_2}{\partial w_2}$$

$$\frac{\partial e}{\partial w_1} = \frac{\partial e}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

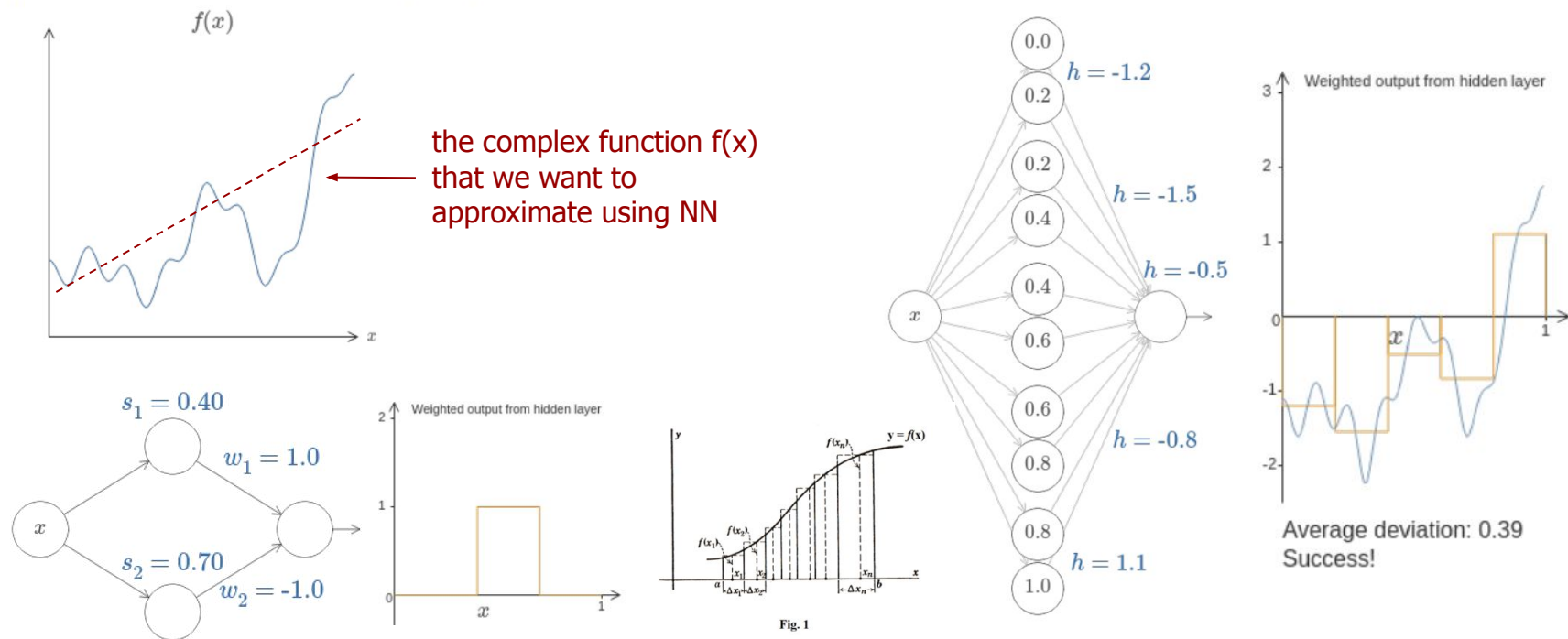
$$w_2 \leftarrow w_2 - \eta \frac{\partial e}{\partial w_2}, \quad w_1 \leftarrow w_1 - \eta \frac{\partial e}{\partial w_1}$$

Universal Approximation Theorem

The expressivity of neural networks - 1 layer

<https://mitliagkas.github.io/ift6085-2020/ift-6085-lecture-10-notes.pdf>

The universal approximation theorem states that any continuous function $f : [0, 1]^n \rightarrow [0, 1]$ can be approximated arbitrarily well by a neural network with at least 1 hidden layer with a finite number of weights, which is what we are going to illustrate in the next subsections.



The expressivity of neural networks

The NN in last page is still linear, how about using sigmoid (Optional)

Cybenko Approximation by Superposition of Sigmoidal Function

$$\sigma(x) \rightarrow \begin{cases} 1 & \text{as } x \rightarrow +\infty \\ 0 & \text{as } x \rightarrow -\infty \end{cases}$$

Theorem 1. Let $C([0, 1]^n)$ denote the set of all continuous function $[0, 1]^n \rightarrow \mathbb{R}$, let σ be any sigmoidal activation function then the finite sum of the form $f(x) = \sum_{i=1}^N \alpha_i \sigma(\mathbf{w}_i^\top x + b_i)$ is dense in $C([0, 1]^n)$

Informally, this theorem is saying that for any $g \in C([0, 1]^n)$ and any $\epsilon > 0$, there exists $f : x \rightarrow \sum_{i=1}^N \alpha_i \sigma(\mathbf{w}_i^\top x + b_i)$ such that $|f(x) - g(x)| < \epsilon$ for all $x \in [0, 1]^n$. continuous function linear combination of finite sigmoid function

Kolmogorov-Arnold Representation theorem

Theorem 2. Any continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ can be written as

$$f(x) = f(x_1, \dots, x_n) = \sum_{q=1}^{Z_m} \phi_q \left(\sum_{q=1}^m \Psi_q(x_q) \right)$$

This implies, among other things, that if we could choose the non-linearity of each unit we can represent any continuous function **exactly** with a NN with 1 hidden layer.

The expressivity of neural networks

How about another direction - deep (Optional)

Lu's Universal Approximation Theorem for Width-Bounded ReLU Network

Theorem 3. (Universal Approximation Theorem for Width-Bounded ReLU Networks). For any Lebesgue-integrable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and any $\epsilon > 0$, there exists a fully-connected ReLU network A with width $d_m \leq n + 4$, such that the function F_A represented by this network satisfies

with fixed width, add deep also works

$$\int_{\mathbb{R}^n} |f(x) - F_A(x)| dx < \epsilon$$

This theorem states that any continuous function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be approximated by a deep ReLU network with width $\leq n + 4$.

Theorem 4. Let n be the input dimension. For any integer $k \geq n + 4$ there exists $F_\alpha : \mathbf{R}^n \rightarrow \mathbf{R}$ represented by a relu neural network α with width $d_m = 2k^2$ and depth $h = 3$ such that for any constant $b > 0$, there exists an $\epsilon > 0$ and for any function $F_\beta : \mathbf{R}^n \rightarrow \mathbf{R}$ represented by a ReLU neural network β whose parameters are bounded in $[-b, b]$ with width $d_m \leq k^{\frac{3}{2}}$ and depth $h \leq k + 2$ the following inequality holds

you can't make it too width but rely only on deep
$$\int_{\mathbf{R}} |F_\alpha - F_\beta| dx \geq \epsilon$$

This theorem states that there are networks such that reducing width requires increasing in the size to compensate, which is similar to that of depth qualitatively.

OK then how to pick a right Dim / N_layer

computation matters, widen takes quadratic time (optional)

Aspect	Widen ($d \rightarrow \alpha d$)	Deepen ($N \rightarrow \beta N$)
Params	$\times \alpha^2$ (quadratic blowup)	$\times \beta$ (linear)
Train FLOPs	$\times \alpha^2$ (short seq) or $\times \alpha$ (long seq)	$\times \beta$
Inference cost	Projections $\times \alpha^2$, latency $\uparrow$	Linear in N
Memory	Params & optimizer $\times \alpha^2$; KV cache $\times \alpha$	All $\times \beta$ (linear)
Parallelism	Better tensor-parallel	Better pipeline-parallel
Infra tradeoff	Bandwidth & memory heavy	Cheaper, stable scaling

Llama 3 - 8B:

$d = 4096$, $N = 32$, $V = 128k$

Embedding = $V * D = 525m$

Layers = N

For each Layer:

Attention QO = $D * D = 33m$

Attention KV = $D/4 * D = 8m$

MLP = $(D * 3D) * 3 = 176m$

RMS_Norm = $D * 2 = 8k$

Sum(layer) = 218m

Sum = $V * D + N(5/4 * D^2 + 3D^2) + 2D + V * D$

Sum = $O(VD + N * D^2 + D)$

Amir's Pre - Crash Narratives

ML expand imagination - compare narrative

$\forall c_i \in C: v_i = \text{Doc2Vec}[c_i]$ c_i is an article, the **Doc2Vec** projects the article into **embedding** v_i .

- Questions to Answer:
 - Do narratives and the similarity of news to past events influence current investor beliefs?
 - Can cash narratives predict market outcomes such as return or volatility?

Calculate cosine similarity between the crash/baseline vs target_doc (Rolling Window Approach, Doc2Vec each month)

Crash - crash-period article

The Aggregated Embedding of:

The Wall Street Journal articles between
[Oct 20 to Oct 23] 1987

Baseline - near crash-period article

The Aggregated Embedding of:

The Wall Street Journal articles between
[Oct 5 to Oct 9] 1987

Target - other articles

The Embedding of:

Another article from WSJ

Then for any target article:

- score1 = similarity to Crash_Vector_87
- score2 = similarity to Baseline_Vector_87
- Final = $\ln(1+\text{score1}) - \ln(1+\text{score2})$

The subtraction controls for the fact that articles might be similar to 1987 WSJ writing style in general. You only want the EXTRA similarity to crash-period writing beyond normal 1987 writing.

Take mean to get **87 narrative index (media panic score)** of a day

From Encoding to Embedding

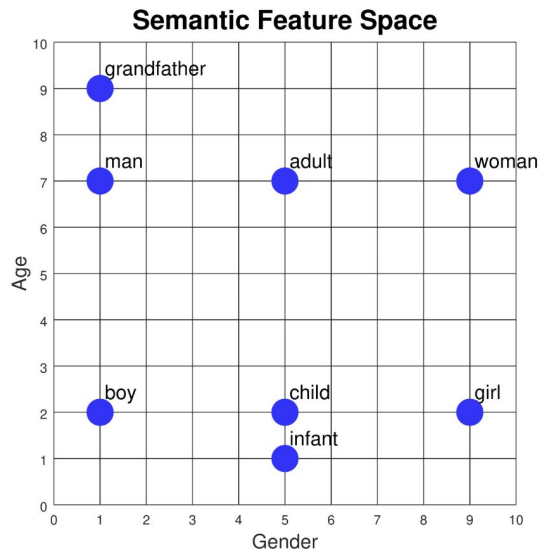
Ways to make text → numeric features

Feature (Color)
Red
Green
Yellow
Green
Red

One Hot Encoding

Red	Green
1	0
0	1
0	0
0	1
1	0

Yellow Column dropped to avoid the Dummy Variable Trap



Word Coordinates			
	Gender	Age	
grandfather	[	1,	9]
man	[	1,	7]
adult	[	5,	7]
woman	[	9,	7]
boy	[	1,	2]
child	[	5,	2]
girl	[	9,	2]
infant	[	5,	1]

“One hot encoding”, known for categorical data processing

- Static
- Sparse data representation - [0, 0, 0, 1, 0, 0]
- High Dimension - 3 unique cate → 3 more dimension
- **Every category is orthogonal to every others**
- You may hear of N-gram, and TF-IDF

Embedding, places categories in low-dimensional geometric space, and categories are **NOT orthogonal**.

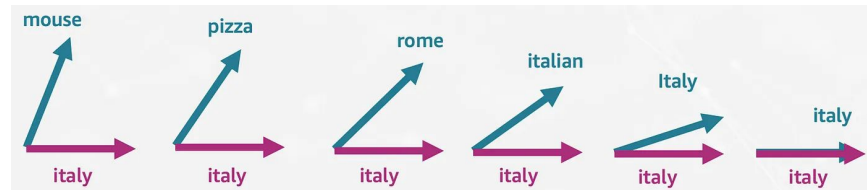
- Learnable, the dimension is fixed.
- Semantic Feature Space → similarity(word1, word2)
- Dense data representation
- Pre-trained, weight freezed, $f(\text{word}) = [\text{embedding}]$.
- We also have $f(\text{sentence}) = [\text{embedding}]$

Embedding

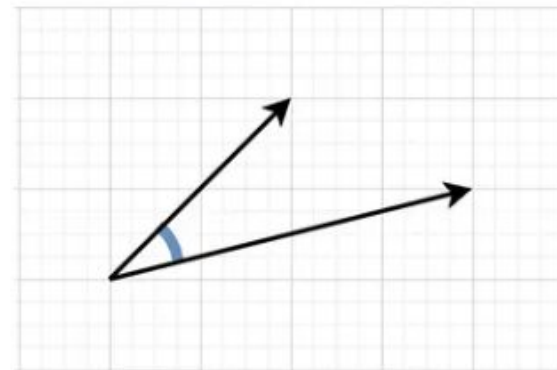


Analogy: Embeddings capture relational meaning!

$\text{vector}('king') - \text{vector}('man') + \text{vector}('woman') = \text{vector}('queen')$
 $\text{vector}('Paris') - \text{vector}('France') + \text{vector}('Italy') = \text{vector}('Rome')$



Cosine Similarity



$$\text{Cosine Similarity} = \frac{\vec{x} \cdot \vec{y}}{||\vec{x}|| \times ||\vec{y}||}$$

Geoeconomic Pressure

An example of LLM

Goal

This paper measures "geoeconomic pressure" - when governments use economic tools (tariffs, sanctions, export controls) to pressure other countries.

Data:

1. Earnings call transcripts (companies talking to investors)
 2. Analyst reports (from JP Morgan, Fitch)
- Total: about 800,000 documents from 2008-2025

STAGE 1: FILTER

Run every document through an LLM (Llama 3.3, 70B parameters) with a basic prompt asking: Does this document discuss tariffs, sanctions, export controls, boycotts, investment screening, or subsidies?

Output: 7 boolean flags (0 or 1 for each policy type)

EXAMPLE: NVIDIA earnings call November 2022 mentions US export controls on selling advanced chips to China.

- Sender: USA
- Receiver: China
- Products: A100 and H100 chips
- Firm response: Supply chains away from China, sales not affected (sold alternative products), overall negative impact

Stage 1:

Is the document relevant?

LLM: YES/NO

Stage 2:

Extract meta information

LLM: YES/NO

Geoeconomic Pressure

Stage 2 (52 fields) and Validation

1. BASIC CLASSIFICATION

- Is firm affected by tariffs at all?
- Are these current tariffs or future/threatened tariffs?

2. WHO/WHAT

- Which countries are imposing the tariffs?
- Which products the firm sells are being tariffed?

3. INVESTMENT

- Is investment going up or down?
- In which countries is investment changing?

4. LABOR/EMPLOYMENT

- Is firm hiring/firing workers?
- Where geographically is employment changing?

Stage 2:

- 36 binary questions (y/n)
 - 16 identity questions (who)
- + 300 word analysis
+ 100 word summary

Validation:

Run all 20 prompts × 3 models
= 60 versions per document
Measure consistency - 78%

1. MODEL PERTURBATION Run same documents through 3 different LLMs:

- Llama 3.3 (70B parameters) - baseline
- Gemma 3 (27B parameters)
- Qwen 2.5 (72B parameters)

Compare if they give similar answers.

2. PROMPT PERTURBATION Create 20 different versions of each prompt:

- 10 minor variations (change few sentences)
- 5 medium variations (different wording, same structure)
- 5 major variations (deliberately "break" the prompt with minimal instructions)

Geoeconomic Pressure

Mostly descriptive, here the model part

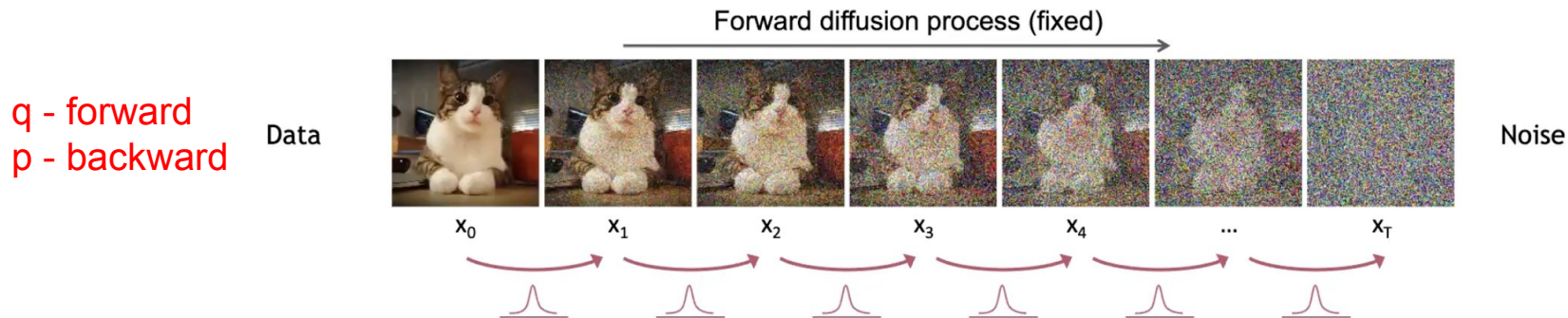
Question: Does US target sectors where China is most vulnerable?

Method - Regression:

- Dependent variable: Probability US firm faces export controls on China (0 or 1)
- Independent variable: "Power" measure = how much damage cutting that sector would do to China
 - $\text{Power} = f(\text{China's spending on that sector, China's import share from US, elasticity of substitution})$

Finding: YES. US systematically uses sectors where China has high dependence and few alternatives.

Diffusion Models



$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}), \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad (2)$$

scale it down (mu) and add noise (var)

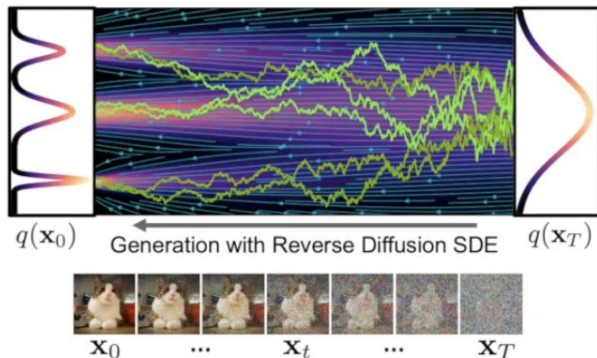
The approximate posterior $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ is fixed to a **Markov chain** that gradually adds Gaussian noise to the data according to a **variance schedule** $\beta_1, \dots, \beta_T$:

Q: What is $\mathbf{x}_{1:T}$? Answer here (Markovian):

$$q(\mathbf{x}_{1:3} | \mathbf{x}_0) = \prod_{t=1}^3 q(\mathbf{x}_t | \mathbf{x}_{t-1}) = q(\mathbf{x}_1 | \mathbf{x}_0)q(\mathbf{x}_2 | \mathbf{x}_1)q(\mathbf{x}_3 | \mathbf{x}_2)$$

Sampling from “Continuous-Time” Diffusion Models

How to solve the generative SDE or ODE in practice?



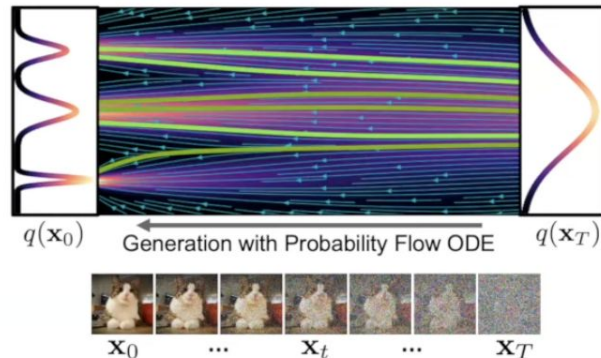
Generative Diffusion SDE:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t) [\mathbf{x}_t + 2\mathbf{s}_\theta(\mathbf{x}_t, t)] dt + \sqrt{\beta(t)} d\bar{\omega}_t$$

➡ *Euler-Maruyama:*

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \frac{1}{2}\beta(t) [\mathbf{x}_t + 2\mathbf{s}_\theta(\mathbf{x}_t, t)] \Delta t + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

➡ *Ancestral Sampling* (Part 1) is also a generative SDE sampler!



Probability Flow ODE:

$$d\mathbf{x}_t = -\frac{1}{2}\beta(t) [\mathbf{x}_t + \mathbf{s}_\theta(\mathbf{x}_t, t)] dt$$

➡ *Euler's Method:*

$$\mathbf{x}_{t-1} = \mathbf{x}_t + \frac{1}{2}\beta(t) [\mathbf{x}_t + \mathbf{s}_\theta(\mathbf{x}_t, t)] \Delta t$$

In practice: Higher-Order ODE solvers

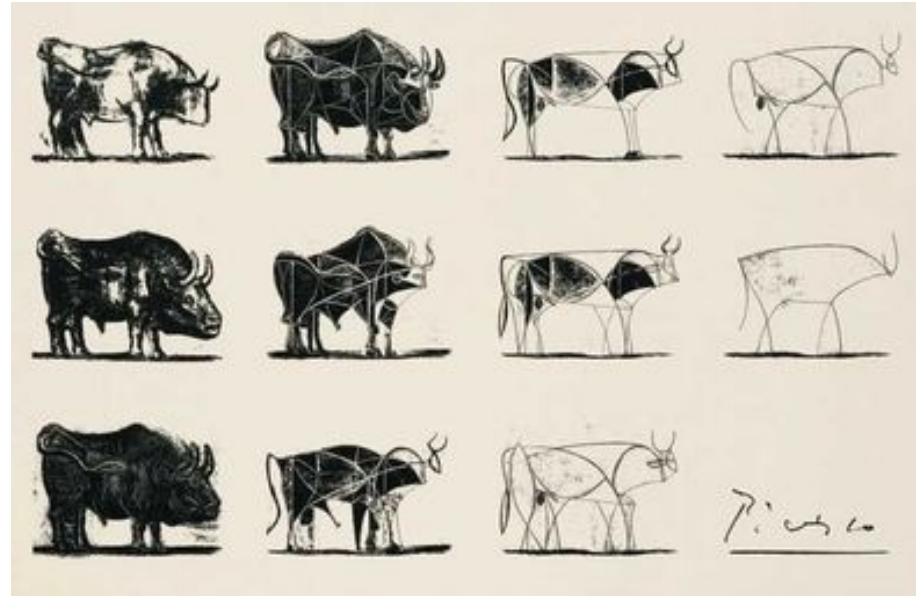
➡ (*Runge-Kutta, linear multistep methods, exponential integrators, ...*)

A poll: which philosophy is better?

Engineer vs Mathematician



Lego Megatron



Picasso's Bull